

Comprendre BBEEasy

Ce document explique le projet BBEEasy pour quelqu'un qui découvre la base de code.

L'objectif est de répondre à trois questions simples:

- Qu'est-ce que fait l'application ?
- Comment le backend et le frontend communiquent ?
- Quel est le rôle des dossiers et des fichiers principaux ?

Le projet est une application web complète autour de BigBlueButton.

Elle permet de gérer des utilisateurs, des salles, des presets, des labels, des enregistrements, des paramètres, l'authentification et la réinitialisation du mot de passe.

Vue d'ensemble

L'application est divisée en trois grandes parties:

- bbbeasy - frontend: interface web React
- bbbeasy - backend: API PHP Fat-Free Framework
- bbbeasy - docs: documentation du projet avec Docusaurus

Le navigateur parle au backend via HTTP.

Le backend utilise PostgreSQL pour les données, et une session PHP pour savoir si l'utilisateur est connecté.

En résumé:

1. Le frontend affiche les pages et envoie les requêtes.
2. Le backend vérifie les données, lit ou écrit dans la base, puis répond en JSON.
3. La session et le cookie PHPSESSID gardent l'utilisateur connecté.

Arborescence générale

bbbeasy - backend/

Le backend contient:

- app/: le vrai code métier
- db/: les migrations de base de données
- public/: le point d'entrée web
- logs/: les fichiers de logs
- tests/: les tests

- `tools/`: quelques scripts utilitaires

bbeasy-frontend/

Le frontend contient:

- `src/`: le code React/TypeScript
- `public/`: les assets publics
- `build/`: le résultat de compilation
- `cypress/`: les tests end-to-end

bbeasy-docs/

La documentation officielle du projet est construite avec Docusaurus.
Elle contient déjà des guides utilisateur, administrateur et développeur.

Comment l'application démarre

Côté frontend

Le point d'entrée principal est `bbeasy-frontend/src/index.tsx`.

Rôle:

- choisit le mode web ou installer
- charge le bon CSS
- monte le composant App
- injecte le routeur React

Le composant principal est `bbeasy-frontend/src/App.tsx`.

Rôle:

- lit l'utilisateur et la session depuis `localStorage`
- remet l'état connecté dans le contexte React
- charge les données utiles selon les permissions
- affiche le header, le sider et les pages

Côté backend

Le point d'entrée web est `bbeasy-backend/public/index.php`.

Rôle:

- initialise le projet

- charge Bootstrap
- démarre l'application Fat-Free Framework

La classe `bbbeasy-backend/app/src/Application/Bootstrap.php` prépare le runtime.

Rôle:

- charge la configuration
- crée la connexion base de données
- prépare la session
- charge les routes
- applique les règles d'accès
- configure les headers CORS

Technologies utilisées

Frontend

- React 17: interface utilisateur
- TypeScript: typage du code
- Ant Design: composants visuels
- react-router-dom: navigation entre pages
- Axios: appels HTTP vers l'API
- i18next et react-i18next: traduction
- pino: logs côté navigateur
- craco: configuration avancée de React Scripts
- react-hot-loader: rechargement rapide en développement

Backend

- PHP: logique serveur
- Fat-Free Framework: framework principal
- PostgreSQL: base de données
- Phinx: migrations SQL
- Monolog: logs applicatifs
- Tracy: gestion d'erreurs
- BigBlueButton API PHP: intégration BigBlueButton
- nette/Utils, symfony/yaml, Linf, sukarix: bibliothèques de support

Outils de développement

- Docusaurus: documentation
- Cypress: tests end-to-end
- ESLint: qualité du code frontend
- Prettier: formatage frontend
- PHP-CS-Fixer: formatage backend
- Composer: dépendances PHP
- Yarn: dépendances frontend
- PM2: exécution/gestion de processus
- Docker: environnement d'exécution

Le backend en détail

1. Configuration

Les fichiers importants sont dans `bbbeasy-backend/app/config/`.

- `default.ini`: valeurs par défaut du framework
- `config-development.ini`: configuration pour développement
- `config-production.ini`: configuration de production
- `config-test.ini`: configuration de test
- `routes.ini`: toutes les routes HTTP
- `access.ini`: règles d'accès aux routes
- `smtp.ini`: configuration mail
- `upload.ini`: paramètres de téléversement
- `client.ini`: configuration côté client / framework

Ce qu'il faut retenir:

- `db.dsn`, `db.username`, `db.password` définissent PostgreSQL
- `session.table = users_sessions` dit où stocker les sessions
- `log.level` règle la verbosité des logs
- `LOGS = ../logs/` indique le dossier des fichiers de log

2. Point d'entrée et bootstrap

`bbbeasy-backend/app/src/Application/Boot.php`

- lit l'environnement
- charge la config client
- crée la connexion SQL
- prépare la session PHP

- configure les logs
- lance le framework

`bbbeasy-backend/app/src/Application/Bootstrap.php`

- hérite de `Boot`
- ajoute la configuration mail
- configure les erreurs
- charge les routes API
- charge les règles ACL
- expose les headers CORS
- autorise dynamiquement les routes selon le rôle

3. Gestion des sessions

Le fichier central est `bbbeasy-backend/app/src/Core/Session.php`.

Rôle:

- stocker les infos de session dans la base
- dire si l'utilisateur est connecté
- écrire les informations de l'utilisateur dans la session
- vider la session au logout
- générer et vérifier les tokens CSRF

Points importants:

- la session contient `user.id`, `user.role`, `user.roleId`, `user.username`, `user.email`, `user.loggedIn`
- `authorizeUser()` marque la session comme connectée
- `revokeUser()` efface la session
- `validateToken()` vérifie le CSRF sur les requêtes POST

4. Contrôle d'accès

Le backend utilise un système d'ACL.

Le fichier `bbbeasy-backend/app/config/access.ini`:

- bloque tout par défaut
- autorise seulement certaines routes publiques
- laisse les autres routes être validées ensuite par le rôle utilisateur

La méthode `allowRoutesDynamically()` dans `Bootstrap.php`:

- lit le rôle de la session
- charge les permissions du rôle depuis la base
- autorise les routes correspondantes

Donc il y a deux niveaux:

- niveau 1: route publique ou non
- niveau 2: permission exacte selon le rôle

5. Les actions backend

Le dossier `bbbeasy-backend/app/src/Actions/` contient les contrôleurs HTTP.

`Actions/Base.php`

Le contrôleur parent de presque toutes les actions.

Il sert à:

- charger la session et l'ACL
- lire le header `Authorization` si besoin
- vérifier les credentials API
- vérifier la session et le CSRF avant les routes POST
- rendre du JSON, du texte ou du XML
- valider des règles communes sur les données

`Actions/Account/`

Ces fichiers gèrent tout ce qui concerne l'utilisateur.

- `Login.php`: connexion
- `Register.php`: inscription
- `Logout.php`: déconnexion
- `ResetPassword.php`: demande de réinitialisation
- `GetResetPasswordToken.php`: vérifie si un token est valide
- `ChangePassword.php`: change le mot de passe avec le token
- `Edit.php`: modification du compte
- `SetLocale.php`: changement de langue

Le login est le flux le plus important:

1. le frontend envoie email + mot de passe
2. le backend valide le format
3. le backend charge l'utilisateur par email
4. il vérifie que le compte est active

5. il compare le mot de passe avec password_verify
6. il remplit la session
7. il renvoie user et session

Actions/Core/

- Main.php: route de base /api
- GetFile.php: accès à un fichier
- GetLocale.php: récupération d'une langue
- Install.php: logique d'installation
- HVSocket.php: composant websocket / communication BigBlueButton

Actions/Users/

Gestion des utilisateurs administratifs.

- Index.php: liste
- Add.php: ajout
- Edit.php: modification
- Delete.php: suppression logique
- Collect.php: récupération formatée

Actions/Rooms/

Gestion des salles de visioconférence.

- Index.php: liste des salles d'un utilisateur
- Add.php: création
- Edit.php: renommage
- Start.php: démarrage d'une salle
- Delete.php: suppression
- View.php: affichage / accès par lien

Actions/Presets/

Gestion des modèles de salle.

- Index.php: liste
- Add.php: ajout
- Edit.php: renommage
- EditSubcategories.php: modification des sous-options
- Copy.php: duplication
- Delete.php: suppression

Actions/Labels/

- Index.php: liste
- Add.php: ajout
- Edit.php: modification
- Delete.php: suppression

Actions/Roles/

- Index.php: liste des rôles
- Add.php: création
- Edit.php: édition
- Delete.php: suppression
- Collect.php: récupération des rôles

Actions/RolesPermissions/

- Collect.php: liste des permissions par rôle

Actions/Settings/

- Collect.php: lire les paramètres
- Edit.php: modifier les paramètres
- SaveLogo.php: enregistrer le logo

Actions/Recordings/

- Collect.php: liste globale
- Index.php: enregistrements d'une salle
- Edit.php: modifier un enregistrement
- Publish.php: publier / dépublier
- Delete.php: suppression

Actions/Logs/

- Collect.php: lire les logs
- Clean.php: nettoyer les logs

Actions/Notification/

- WarningNotification.php: alerte liée aux notifications

6. Les modèles

Le dossier bbbeasy-backend/app/src/Models/ contient les objets métier liés à la base.

Les principaux fichiers:

- User.php: utilisateur, mot de passe, statut, permissions, connexion

- `UserSession.php`: lecture de la session stockée en base
- `ResetPasswordToken.php`: jetons de reset mot de passe
- `Role.php`: rôle et permissions
- `Room.php`: salle BigBlueButton
- `Preset.php`: modèles de salle
- `Setting.php`: paramètres globaux
- `Label.php`: étiquettes
- `RoomLabel.php`: lien salle/label
- `RolePermission.php`: lien rôle/permission
- `PresetSetting.php`: paramètres associés à un preset

Idée importante:

- les modèles ne font pas seulement du SQL
- ils encapsulent aussi de la logique métier
- par exemple `User::verifyPassword()` ou `User::saveUserWithDefaultPreset()`

7. Les enums

Le dossier `bbbeasy-backend/app/src/Enum/` donne des constantes lisibles.

Exemples:

- `UserStatus.php`: `active`, `inactive`, `pending`, `deleted`
- `UserRole.php`: `visitor`, `lecturer`, `administrator`, `api`
- `ResetTokenStatus.php`: `new`, `consumed`, `expired`

Ces classes évitent d'écrire des chaînes de caractères partout dans le code.

8. Les utilitaires

Le dossier `bbbeasy-backend/app/src/Utils/` contient des fonctions transverses.

Fichiers importants:

- `Environment.php`: détermine l'environnement
- `SecurityUtils.php`: règles de sécurité sur les mots de passe
- `PresetProcessor.php`: transforme les données des presets en paramètres BigBlueButton
- `PrivilegeUtils.php`: aide sur les droits

Le dossier `bbbeasy-backend/app/src/Validation/` gère les contrôles de données.

- `DataChecker.php`: accumule les erreurs de validation

Le dossier `bbbeasy-backend/app/src/Mail/` gère les mails.

Le dossier `bbbeasy-backend/app/src/Log/` gère l'écriture des logs.

9. Les migrations

Le dossier `bbbeasy-backend/db/migrations/` contient l'évolution de la base.

On y trouve notamment:

- création des tables utilisateurs
- création des sessions
- création des tokens de reset mot de passe
- ajout de colonnes comme `expires`

Rôle des migrations:

- créer la structure SQL
- faire évoluer la base proprement
- garder l'historique des changements

10. Les logs backend

Le backend écrit les logs dans `bbbeasy-backend/logs/`.

Le mécanisme est géré par:

- `bbbeasy-backend/app/src/Log/LogWriterTrait.php`
- `bbbeasy-backend/app/src/Application/Boot.php`
- `bbbeasy-backend/app/src/Application/Bootstrap.php`

Ce qu'il faut comprendre:

- `Boot.php` définit `application.logfile`
- `LogWriterTrait.php` crée un logger Monolog avec un `StreamHandler`
- les logs changent selon le jour
- `error_log` PHP est aussi redirigé vers un fichier dédié

En pratique, tu peux y voir:

- les connexions
- les erreurs
- les accès refusés
- les opérations métier
- les traces de session SQL si `log.session = true`

Le frontend en détail

1. Point d'entrée

`bbbeasy-frontend/src/index.tsx`

Rôle:

- démarre React
- choisit entre l'application web et l'installer
- injecte les routes
- charge le style global adéquat

2. Application principale

`bbbeasy-frontend/src/App.tsx`

Rôle:

- reconstruit l'état connecté depuis `localStorage`
- hydrate le contexte utilisateur
- charge des données de départ selon les permissions
- affiche l'en-tête, le menu latéral, le contenu et le pied de page

Il y a deux contextes importants:

- `UserContext`: état de connexion et utilisateur courant
- `DataContext`: données métiers partagées entre composants

3. Configuration des routes

Le dossier `bbbeasy-frontend/src/routing/` contient la navigation.

Fichiers importants:

- `backend-config.tsx`: construit les URLs de l'API backend
- `Router.tsx`: affiche la bonne page selon l'URL
- `config.ts / config-install.ts`: listes de routes web ou installateur

Le routeur backend est construit à partir de:

- `window.location.origin`
- `process.env.REACT_APP_API_ROUTE`

Donc le frontend sait où appeler l'API sans hardcoder le domaine.

4. Services

Le dossier `bbbeasy-frontend/src/services/` contient la couche d'accès à l'API.

Chaque service correspond à un domaine métier:

- `auth.service.ts`: login, register, logout, reset password, edit account
- `users.service.ts`: utilisateurs
- `rooms.service.ts`: salles
- `labels.service.ts`: labels
- `roles.service.ts`: rôles
- `presets.service.ts`: presets
- `preset.settings.service.ts`: paramètres de preset
- `settings.service.ts`: paramètres globaux
- `recordings.service.ts`: enregistrements
- `notification.service.ts`: notifications
- `locale.service.ts`: langue
- `install.service.ts`: installation

Le service d'authentification est central.

Il:

- envoie les requêtes `/account/*`
- stocke l'utilisateur et la session dans `localStorage`
- relit ensuite ces données au chargement

5. Dossier lib/

`bbbeasy-frontend/src/lib/` contient des briques techniques partagées.

Fichiers importants:

- `AxiosInstance.ts`: instance Axios avec `withCredentials = true`
- `AuthService.ts` n'existe pas ici, la logique est dans `services/auth.service.ts`
- `AuthHeader.tsx`: construit un header Cookie à partir de la session locale
- `Logger.tsx`: logger frontend basé sur pino
- `UserContext.tsx`: contexte utilisateur
- `RoomsContext.tsx`: contexte pour rooms, labels, presets

Le point important:

- le navigateur envoie les cookies automatiquement
- certaines requêtes peuvent aussi reconstruire un header de cookie
- les données de session sont gardées localement pour l'UI

6. Composants d'authentification

Le dossier `bbbeasy-frontend/src/components/auth/` contient 4 écrans.

- `Login.tsx`: écran de connexion
- `Register.tsx`: inscription
- `ResetPassword.tsx`: demande de reset email
- `ChangePassword.tsx`: saisie d'un nouveau mot de passe avec token

Rôle de chacun:

- `Login.tsx` envoie email + mot de passe et reçoit user + session
- `Register.tsx` envoie le formulaire d'inscription
- `ResetPassword.tsx` demande l'envoi d'un lien email
- `ChangePassword.tsx` vérifie le token et change le mot de passe

7. Layout

Le dossier `bbbeasy-frontend/src/components/layout/` contient la structure visuelle commune.

- `AppHeader.tsx`: barre du haut, profil, logout, langue, recherche
- `AppSider.tsx`: menu latéral, actions rapides, navigation selon permissions
- `AppFooter.tsx`: pied de page

Ces composants dépendent de l'état connecté.

8. Garde de routes

- `PrivateRoute.tsx`: protège les pages privées
- `PublicRoute.tsx`: empêche de revenir à login/register si déjà connecté

Ces composants ne font pas une vraie validation serveur.

Ils vérifient surtout la présence de user et session dans `localStorage`.

9. Types

Le dossier `bbbeasy-frontend/src/types/` définit les formes des données.

Exemples:

- `UserType.ts`: structure d'un utilisateur
- `SessionType.ts`: `PHPSESSID` + expiration
- `RoomType.ts`, `LabelType.ts`, `PresetType.ts`, etc.

Intérêt:

- éviter les erreurs de structure

- rendre le code plus lisible
- faciliter l'autocomplétion

10. Traductions

Le dossier `bbbeasy-frontend/src/locale/` contient les fichiers de traduction.

Exemples:

- `en-US.json`
- `fr-FR.json`
- `ar-TN.json`

Le frontend utilise `i18next` pour afficher l'interface dans plusieurs langues.

11. Fonctions utilitaires

Le dossier `bbbeasy-frontend/src/functions/` contient des petites fonctions réutilisables.

Exemple:

- comparaison
- génération de chaînes
- aides UI diverses

12. Logs frontend

Le fichier `bbbeasy-frontend/src/lib/Logger.tsx` crée un logger `pino`.

Rôle:

- loguer les événements côté navigateur
- transmettre certains logs au backend via `POST /api/logs`
- garder une trace des erreurs ou informations de navigation

Donc le frontend ne logue pas seulement dans la console:

- il peut aussi remonter des événements au serveur

Focus sur l'authentification

Connexion

Fichiers impliqués:

- `bbbeasy-frontend/src/components/auth/Login.tsx`

- bbbeasy-frontend/src/services/auth.service.ts
- bbbeasy-backend/app/src/Actions/Account/Login.php
- bbbeasy-backend/app/src/Core/Session.php
- bbbeasy-backend/app/src/Models/User.php

Flux:

1. L'utilisateur remplit email et mot de passe.
2. Le frontend appelle POST /api/account/login.
3. Le backend vérifie le format des champs.
4. Il charge l'utilisateur avec l'email.
5. Il vérifie le statut du compte.
6. Il compare le mot de passe avec le hash.
7. Il appelle authorizeUser().
8. Il renvoie user + session.
9. Le frontend sauvegarde les deux objets dans localStorage.
10. L'interface passe en mode connecté.

Déconnexion

Fichiers:

- bbbeasy-frontend/src/components/layout/AppHeader.tsx
- bbbeasy-frontend/src/services/auth.service.ts
- bbbeasy-backend/app/src/Actions/Account/Logout.php
- bbbeasy-backend/app/src/Core/Session.php

Flux:

1. le frontend appelle GET /api/account/logout
2. le backend vide la session
3. le frontend supprime user et session du localStorage
4. la page retourne à la connexion

Inscription

Fichiers:

- bbbeasy-frontend/src/components/auth/Register.tsx
- bbbeasy-backend/app/src/Actions/Account/Register.php
- bbbeasy-backend/app/src/Models/User.php

Flux:

1. le formulaire collecte username, email, mot de passe et confirmation
2. le backend valide les contraintes
3. il vérifie que les identifiants ne sont pas déjà utilisés
4. il crée un compte en pending
5. il crée un preset par défaut

Réinitialisation du mot de passe

Fichiers:

- bbbeasy-frontend/src/components/auth/ResetPassword.tsx
- bbbeasy-frontend/src/components/auth/ChangePassword.tsx
- bbbeasy-backend/app/src/Actions/Account/ResetPassword.php
- bbbeasy-backend/app/src/Actions/Account/GetResetPasswordToken.php
- bbbeasy-backend/app/src/Actions/Account/ChangePassword.php
- bbbeasy-backend/app/src/Models/ResetPasswordToken.php

Flux:

1. l'utilisateur demande un reset par email
2. le backend crée un token
3. il stocke le token avec une expiration
4. il envoie un mail avec le lien
5. l'écran ChangePassword vérifie le token
6. le backend accepte le changement si le token est valide
7. le token passe à l'état consumed
8. le compte repasse active

Données et base de données

Le projet utilise PostgreSQL.

Les tables importantes sont notamment:

- users
- users_sessions
- reset_password_tokens
- roles
- roles_permissions
- rooms

- labels
- presets
- settings

Les migrations dans `db/migrations/` expliquent comment ces tables ont été créées et modifiées.

Le backend utilise aussi la table `users_sessions` pour lier la session à l'utilisateur.

Logs et fichiers de suivi

Logs serveur

Répertoire:

- `bbbeasy-backend/logs/`

Ce dossier contient:

- les logs applicatifs
- les erreurs PHP
- les logs de session SQL si activés

Nom des fichiers:

- `app-YYYY-MM-DD.log`
- `app-error-YYYY-MM-DD.log`
- éventuellement des fichiers spécifiques comme `smtp-YYYY-MM-DD.log`

Logs navigateur

Le frontend peut envoyer certains logs au backend.

Le but est d'avoir une meilleure visibilité sur ce qui se passe dans l'interface.

Comment lire ce projet quand on débute

Si tu veux l'apprendre sans te perdre, lis dans cet ordre:

1. `bbbeasy-frontend/src/index.tsx`
2. `bbbeasy-frontend/src/App.tsx`
3. `bbbeasy-frontend/src/components/auth/Login.tsx`
4. `bbbeasy-frontend/src/services/auth.service.ts`
5. `bbbeasy-backend/public/index.php`
6. `bbbeasy-backend/app/src/Application/Bootstrap.php`
7. `bbbeasy-backend/app/src/Actions/Base.php`

8. `bbbeasy-backend/app/src/Actions/Account/Login.php`
9. `bbbeasy-backend/app/src/Core/Session.php`
10. `bbbeasy-backend/app/src/Models/User.php`

Ensuite, tu peux regarder les autres dossiers métier:

- Rooms
- Presets
- Labels
- Users
- Roles
- Recordings

Ce qu'il faut retenir

- le frontend n'est qu'une interface
- le backend contient la vraie logique métier
- la session PHP garde l'utilisateur connecté
- `localStorage` sert surtout à l'état UI et au rechargement
- les permissions viennent du rôle de l'utilisateur
- les logs sont répartis entre navigateur et serveur
- la base PostgreSQL stocke les utilisateurs, sessions et objets métier

Conclusion

BBBEasy est une application web assez classique dans sa structure:

- React pour l'interface
- PHP Fat-Free Framework pour l'API
- PostgreSQL pour les données
- sessions PHP pour l'authentification
- logs Monolog/Pino pour le suivi

La partie la plus importante à comprendre au début est le trio:

- `App.tsx` côté frontend
- `Bootstrap.php` et `Base.php` côté backend
- `Session.php` pour la connexion

Si tu comprends ces trois éléments, le reste du projet devient beaucoup plus simple à lire.